

Extended Abstract: *Observability for Containerized CI/CD Services: Improving Reliability*

Eduardo Sousa

Departamento de Engenharia Informática / Mestrado em Engenharia Informática
Instituto Superior de Engenharia do Porto (ISEP)

December 31, 2025

Abstract

Internal CI/CD services (GitLab, Jenkins, Nexus, SonarQube, Dependency-Track) often run in Docker on mixed Linux/Windows hosts without administrative access. In these constrained environments, teams struggle to detect faults quickly and to diagnose root causes because telemetry is fragmented and host-level instrumentation is not permitted. This work proposes a container-first observability architecture that collects metrics, logs and traces from non-privileged containers using sidecar exporters, agent collectors and runtime APIs, and correlates signals in a unified metrics pipeline with dashboards and alerts. The approach is validated on a four-server, on-premise setup and evaluated with MTTD, MTTR, pipeline throughput, alert precision and overhead (target below 5%). The expected contribution is a reference architecture, reproducible prototype and decision guidance that improve CI/CD reliability under strict privilege constraints.

Contents

1	Introduction and Problem Framing	1
2	Research Questions and Objectives	1
3	State of the Art	2
3.1	Literature Search Strategy	2
3.2	Expected Coverage and Research Gap	3
3.3	Preliminary Findings	3
4	Proposed Methodology	5
5	Work Plan	5
6	Ethical and Professional Considerations	7
	References	7

Acronyms

CICD Continuous Integration/Continuous Deployment

GDPR General Data Protection Regulation

IP Internet Protocol

MTTD Mean Time to Detection

MTTR Mean Time to Recovery

PRISMA Preferred Reporting Items for Systematic Reviews and Meta-Analyses

DORA DevOps Research and Assessment

1 Introduction and Problem Framing

Internal Continuous Integration/Continuous Deployment (CI/CD) services (GitLab, Jenkins, SonarQube, Nexus, Dependency-Track) deployed in Docker containers are critical to software delivery pipelines. Reliability issues directly affect development velocity and release cadence. However, constrained deployment contexts, non-privileged containers, no host administrative access, and heterogeneous operating systems (Linux and Windows).

This research examines integrated observability (systematic collection, correlation and analysis of metrics, logs and distributed traces) in such constrained environments. The proposed solution combines non-privileged telemetry collection (sidecar exporters, agent collectors, runtime APIs) with correlation mechanisms to improve Mean Time to Detection (MTTD), Mean Time to Recovery (MTTR), and pipeline throughput while maintaining sub-5% resource overhead. The work is grounded in a real-world, four-server on-premise deployment (three Linux, one Windows) with no administrative access, exemplifying operational constraints typical of internally-managed infrastructure where third-party services deployed via Docker [1] cannot be modified.

The central research problem is: *How can an integrated observability framework be designed and implemented for containerized CI/CD platforms operating under non-privileged access constraints and heterogeneous infrastructure, to measurably improve reliability and reduce incident response times?*

2 Research Questions and Objectives

This work is driven by the central question stated in the Introduction: how an integrated observability framework can be designed and implemented for containerized CI/CD platforms operating under non-privileged access constraints and heterogeneous infrastructure so as to measurably improve system reliability and reduce incident response times.

Sub-questions:

1. How can metrics, logs and distributed traces be collected from containerized CI/CD services when only non-privileged containers are available (no host administrative access)?
2. Which instrumentation and deployment patterns (sidecars, agent collectors, push vs pull pipelines) best support cross-server correlation in mixed OS environments (Linux and Windows) while keeping resource overhead minimal?
3. What is the measurable impact of the proposed observability framework on operational reliability, expressed as improvements in MTTD, MTTR, and pipeline throughput, compared to the current baseline?
4. What practical constraints and trade-offs (data fidelity, retention, network and CPU overhead, security) arise from non-privileged telemetry collection, and how can they be mitigated in a reproducible reference architecture?

Objectives:

- **O1 Reference architecture:** Design a container-first architecture specifying exporters, collectors, storage backends and visualization components deployable without host administrative privileges.

- **O2 Prototype implementation:** Implement a reproducible prototype (container images and deployment templates) that instruments GitLab, Jenkins, SonarQube, Dependency-Track and Nexus in the four-server environment.
- **O3 Quantitative evaluation:** Measure and compare MTTD, MTTR, telemetry pipeline throughput and resource overhead before and after deployment; document measurement methodology for repeatability.
- **O4 Performance constraints:** Maintain resource overhead below an acceptable threshold (target: under 5% of host capacity under normal load); document trade-offs when targets are not met.
- **O5 Reproducibility and guidance:** Provide configuration templates, deployment scripts and decision guidance for adoption in constrained environments.

3 State of the Art

This section outlines the systematic approach to identifying, selecting, and analyzing relevant literature on observability for containerized systems, with particular focus on constrained deployment environments and CI/CD platform reliability.

3.1 Literature Search Strategy

A systematic literature review will be conducted following Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines to ensure comprehensive and reproducible coverage of the research domain. The search strategy encompasses the following elements:

Information sources:

- **Academic databases:** IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, and Google Scholar for peer-reviewed publications
- **Technical repositories:** arXiv for preprints and working papers in systems and software engineering
- **Gray literature:** Technical documentation from established open-source observability projects, industry whitepapers, and practitioner reports from reputable sources (given the rapid evolution of container observability practices)

Search terms and queries: Core search queries will combine terms from three concept groups using Boolean operators:

1. *Observability concepts:* “observability”, “monitoring”, “telemetry”, “instrumentation”, “metrics”, “logging”, “tracing”, “distributed tracing”
2. *Container and deployment context:* “container”, “containerized”, “Docker”, “non-privileged”, “unprivileged”, “sidecar”, “agent”
3. *Application domain:* “CI/CD”, “continuous integration”, “continuous deployment”, “DevOps”, “reliability”, “incident response”, “MTTD”, “MTTR”

Example query structure: (*observability* OR *monitoring* OR *telemetry*) AND (*container* OR *containerized* OR *non-privileged*) AND (*CI/CD* OR *reliability* OR *incident response*)

Inclusion and exclusion criteria:

- **Inclusion:** Peer-reviewed publications from 2018–2025, technical reports and documentation addressing container telemetry collection, instrumentation patterns, performance evaluation, or reliability improvement in constrained or heterogeneous environments, comparative studies of observability approaches, and empirical evaluations of monitoring overhead
- **Exclusion:** Publications exclusively focused on privileged host access patterns without discussion of constraints, cloud-provider-specific solutions that cannot generalize to on-premise infrastructure, purely theoretical work without practical implementation guidance, and papers predating 2018 (unless seminal works establishing foundational concepts)

Selection process: Literature selection will follow a three-stage process: (1) title and abstract screening to identify potentially relevant sources, (2) full-text review to assess relevance against inclusion/exclusion criteria, and (3) quality assessment focusing on methodological rigor, reproducibility, and applicability to constrained environments. Reference chaining (backward and forward citation tracking) will supplement database searches to identify additional relevant work.

3.2 Expected Coverage and Research Gap

The literature review will systematically examine existing approaches to container observability, deployment patterns for telemetry collection (agent-based, sidecar, host-level), instrumentation frameworks and their privilege requirements, performance and overhead studies, and reliability improvements achieved through observability implementations.

Preliminary exploration suggests that while cloud-native observability is well-documented for environments with elevated privileges and homogeneous infrastructure, there is limited empirical guidance for teams operating internal, containerized CI/CD platforms under strict privilege constraints and mixed operating systems.

An additional and frequently overlooked constraint is the fact that many critical services in such environments are packaged, third-party applications (not developed in-house). When teams do not own the service code or container images, they have reduced ability to add instrumentation or change deployment configurations, which narrows feasible telemetry approaches. Consequently, solutions that assume service ownership or unrestricted host access may not be applicable. This research targets that combined constraint set, no host administrative privileges and limited service ownership, and seeks empirical evidence and practical patterns that work within it.

The subsequent section specifies the methodology for technology assessment, prototype development and empirical validation informed by the systematic review.

3.3 Preliminary Findings

Pappula et al. (2021) [2] present a multi-signal observability framework for distributed systems emphasizing correlation of metrics, logs, and distributed traces to improve fault detection and diagnosis. The authors identify representative metrics across multiple

dimensions: latency percentiles (p50, p95, p99), throughput (requests per second), error rates, saturation (resource utilization, queue depth), and business signals. For correlation, they describe a unified telemetry pipeline integrating OpenTelemetry for instrumentation, Prometheus for metrics storage, Jaeger for distributed tracing, and ELK stack for log aggregation. Correlation relies on shared identifiers (trace IDs, request IDs) propagated across service boundaries and embedded in metrics labels, log entries, and trace spans, enabling operators to navigate from metric anomalies to related logs and traces. The authors report empirical improvements in microservices deployments: MTTD reduced by 40–60% through proactive alerting on correlated signals, and MTTR reduced by 30–50% through faster root-cause identification. Operational constraints include telemetry noise requiring alert tuning, storage costs for high-volume trace and log data necessitating sampling strategies, and scalability limits of centralized pipelines requiring distributed collectors. The paper provides foundational concepts for multi-signal correlation (**O1**), validated evaluation metrics (**O3**: MTTD/MTTR), and guidance on instrumentation trade-offs (**O4**). However, the study assumes cloud-native microservices with flexible infrastructure control, Kubernetes orchestration, and service ownership allowing code assumptions that do not hold in constrained deployments with non-privileged containers, no host administrative access, and third-party packaged services.

Siddiqui et al. (2023) [3] provide practical guidance on Jenkins-based CICD pipeline monitoring using Prometheus and Grafana. The authors identify key performance indicators: build duration (by pipeline stage), queue depth and wait time, agent availability and utilization, job success/failure rates, and response time for critical workflows. Integration patterns combine Jenkins plugins (Metrics Plugin, Prometheus Plugin) for metrics exposure, Prometheus scraping at 15–60 second intervals, and Grafana dashboards with alerting. The paper presents dashboard templates for time-series visualizations, histograms, and status tables, plus alerting rules triggered by threshold violations (failure rates, queue times, agent availability). Case studies demonstrate detection of misconfigured agents, resource exhaustion, and plugin incompatibilities. The paper contributes concrete CICD-specific guidance (**O1**: architecture patterns, **O3**: validated KPIs, **O5**: dashboard and alert templates) applicable to Jenkins and similar services (GitLab, Nexus, SonarQube) with plugin ecosystems. However, the approach assumes administrative flexibility for plugin installation and infrastructure access, and does not address non-privileged containers, mixed-OS environments, or sidecar-based instrumentation.

Both papers establish foundations for observability architecture, multi-signal correlation, and CICD monitoring, but assume flexible infrastructure control and service ownership. Neither addresses the combined constraint set of non-privileged containers, no host administrative access, third-party packaged services, and heterogeneous operating systems. The present work extends these frameworks by evaluating instrumentation patterns operating within strict privilege constraints (sidecar exporters, read-only Docker socket access, stdout/stderr log forwarding, environment variable correlation), quantifying impact on MTTD, MTTR, and pipeline throughput (**O3**), and delivering a reproducible reference architecture, configuration templates, and decision guidance (**O1**, **O2**, **O4**, **O5**) for constrained on-premise CICD infrastructure.

4 Proposed Methodology

This research adopts a design science approach [4], combining systematic literature review, comparative technology assessment, architectural design, prototype implementation, and empirical evaluation. Five interconnected phases progress from problem understanding to artifact evaluation:

Phase 1: Problem Analysis and Literature Review. A PRISMA-guided systematic review of observability frameworks, containerized monitoring, and CI/CD reliability will identify existing approaches, their privilege assumptions, and gaps in constrained-environment guidance.

Phase 2: Technology Assessment. Candidate tools (Prometheus [5], Loki, Jaeger, OpenTelemetry) will be evaluated against constraints: non-privileged execution, Linux/Windows support, sub-5% overhead, Docker compatibility. Assessment focuses on deployment complexity, resource consumption, correlation capabilities, and maturity.

Phase 3: Architecture Design. A reference architecture will specify component placement (exporters, collectors, storage, visualization), data flow, correlation mechanisms, and integration with GitLab, Jenkins, SonarQube, Nexus, Dependency-Track. Design artifacts include deployment diagrams, schemas, and configuration templates.

Phase 4: Prototype Implementation. The architecture will be deployed on the four-server containerized infrastructure using sidecars for instrumentation and agent-based collectors within privilege constraints. Configuration and deployment scripts will be version-controlled for reproducibility.

Phase 5: Evaluation and Validation. Quantitative metrics include MTTD, MTTR (primary outcomes), alert precision, detection coverage, time to diagnostic context, telemetry pipeline health, and instrumentation overhead (target: below 5%). Baseline measurements (current reactive approach) will be compared with post-deployment measurements. Controlled fault injection (service degradation, resource exhaustion) will test detection. Alignment with DevOps Research and Assessment (DORA) [6, 7] will focus on change failure rate and recovery performance. Qualitative feedback from DevOps practitioners will assess operational impact.

5 Work Plan

The research follows a two-phase timeline: (1) PREPD course completion (November 2025–January 2026), and (2) thesis development (January–June 2026).

Phase 1: PREPD course deliverable (November 2025–January 2026)

- **Literature review** (November–early January): Conduct a systematic review using PRISMA to identify observability frameworks, containerized monitoring approaches and non-privileged deployment patterns; document the technology landscape and research gaps.
- **Technology assessment** (December–early January): Evaluate candidate technologies (metrics, logs, tracing, visualization) based on literature findings; establish selection criteria aligned with non-privileged container constraints and heterogeneous infrastructure.
- **Planning and milestones** (November–January): Define milestones, deliverables, responsibilities, communication channels and presentation schedule for PREPD

delivery.

- **Initial architecture design** (early January): Design the high-level observability architecture; define component interactions, data flows and deployment patterns; validate against constraints.
- **Baseline data collection** (December–early January): Collect existing logs and lightweight telemetry to establish baseline MTTD, MTTR and pipeline throughput; define measurement methods and retention scope.
- **PREPD delivery** (January 7, 2026): Submit the PREPD report consolidating extended abstract, methodology, state of the art and proposed solution; prepare presentation materials.

Phase 2: Thesis development (January–June 2026)

- **State of the art and technology selection** (January–February): Finalize the systematic review; complete comparative technology assessment and justify the selected stack based on evaluation criteria.
- **Detailed architecture and prototype implementation** (February–March): Refine the reference architecture; implement the prototype observability stack in staging; configure metrics, logs and tracing components; develop deployment automation scripts.
- **Production deployment** (April): Instrument production CI/CD services (GitLab, Jenkins, SonarQube, Dependency-Track, Nexus) with observability agents; deploy to existing infrastructure subject to availability and provisioning approvals.
- **Observability data collection** (May): Collect MTTD, MTTR and pipeline throughput metrics over a four-week period; document incident detection and response under baseline and observability-enabled scenarios.
- **Validation, analysis and thesis writing** (early June): Conduct structured interviews for qualitative validation; analyze quantitative metrics; complete thesis chapters (implementation, evaluation, conclusions).
- **Final thesis submission** (mid-June 2026): Incorporate advisor feedback, perform revisions and submit the dissertation.

Progress monitoring: Bi-weekly meetings with the university advisor (Tuesdays), weekly meetings with the company supervisor (Mondays) and monthly written progress reports. Key milestones: PREPD delivery (January 7), prototype completion (March), production deployment (April), data collection completion (May).

Risk management: Infrastructure provisioning delays are mitigated through extended staging testing to maintain progress independent of production access. Evaluation adapts to staging data if required. Personnel unavailability risks are addressed through flexible scheduling and multiple participant recruitment. Technical risks (tool incompatibilities, performance issues) are mitigated through early proof-of-concept validation in Phase 2. Baseline data quality risks are addressed by establishing minimal logging before formal baseline measurement.

6 Ethical and Professional Considerations

This research includes structured interviews with DevOps and operations personnel to support qualitative validation of the observability framework. Interview protocols and consent forms will be reviewed and approved by the thesis advisor prior to data collection. Participants will provide informed consent; participation is voluntary; responses will be anonymized in reporting and publications. Personally identifiable information will not be disclosed, and interview data will be handled confidentially in accordance with research ethics standards. Recordings and transcripts will be retained for twelve months following thesis submission and then securely deleted. Interview data will not be shared with company management in identifiable form.

The project operates within a company production infrastructure, involving access to operational logs, metrics and system configurations that constitute confidential organizational information. Infrastructure details, service configurations, incident patterns and performance data will be treated as confidential and subject to organizational information security policies. Published results will anonymize hostnames, Internet Protocol (IP) addresses, service names and proprietary configurations. The company supervisor has approved the research scope and publication approach.

Deployment of the observability framework to production systems entails organizational impact and operational risk. All instrumentation will undergo testing in staging prior to production deployment, following established change-management procedures. Performance overhead will be monitored continuously to avoid adverse effects on CICD service availability. Rollback procedures will be prepared for all production changes, and deployment timing will be coordinated with management to minimize disruption.

This research does not involve collection or processing of personal data as defined under General Data Protection Regulation (GDPR), medical or health-related information, vulnerable populations or other regulated data types. Ethical and professional safeguards have been identified and will be implemented as described above.

References

- [1] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux J.*, vol. 2014, Mar. 2014.
- [2] K. K. Pappula, S. Anasuri, and G. P. Rusum, “Building observability into full-stack systems: Metrics that matter,” *International Journal of Emerging Research in Engineering and Technology*, vol. 2, no. 4, pp. 48–58, 2021.
- [3] I. Siddiqui, A. Pandey, S. Jain, H. Kothadia, R. Agrawal, and N. Chankhore, “Comprehensive monitoring and observability with jenkins and grafana: A review of integration strategies, best practices, and emerging trends,” in *2023 7th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT)*, pp. 1–5, Oct 2023.
- [4] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [5] J. Volz, “Prometheus: Monitoring at soundcloud,” *SoundCloud Developers Blog*, 2016.

- [6] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, 2018.
- [7] N. Forsgren, D. Smith, J. Humble, and J. Frazelle, “The 2024 state of devops report,” *DORA (DevOps Research and Assessment)*, 2024.